

CHAPTER I

INTRODUCTION

1.1 OVERVIEW:

In many countries, wearing helmets while riding motorcycles is a legal requirement to ensure the safety of riders. Despite this, the enforcement of helmet laws has been inefficient due to the limitations of manual monitoring by police forces, which are often understaffed and overburdened. CCTV surveillance is used in major cities to address this issue; however, these methods are semi-automated and still rely heavily on human intervention, leading to inconsistencies in enforcement. As motorcycle usage continues to grow, particularly in countries like India, the rate of head injuries and fatalities due to non-compliance with helmet laws has become a pressing concern. This highlights the need for a more efficient, scalable, and automated solution to monitor helmet usage and penalize offenders[8].

To address this challenge, this paper proposes an automated system that leverages machine learning to detect motorcyclists who are not wearing helmets and retrieve their license plate numbers in real time. Using CCTV camera footage from road junctions, the system employs advanced object detection algorithms to identify helmetless riders. Once detected, the license plate of the motorcycle is captured, and Optical Character Recognition (OCR) is used to extract the license plate number. This information can then be used by authorities to issue penalties or warnings to the offenders. By automating this process, the system not only reduces the dependency on human resources but also ensures consistent and accurate enforcement of helmet laws[2].

The integration of machine learning in transportation systems has the potential to significantly enhance road safety. Automated helmet detection and license plate recognition systems serve as a practical example of how technology can be applied to improve compliance with traffic regulations. Beyond enforcement, such systems generate valuable data that policymakers can use to make informed decisions about road safety measures. However, implementing these systems comes with challenges. Factors like lighting conditions, weather, and the distance between the camera and the motorcycle can impact the accuracy of detection and recognition algorithms. Additionally, substantial investments are needed to establish the necessary infrastructure and maintain such systems effectively.

Despite these challenges, the benefits of these automated systems are substantial, particularly in developing countries where road safety remains a critical concern. By reducing manual intervention, improving enforcement, and providing actionable insights, these systems can help reduce fatalities and injuries caused by motorcycle accidents.

Continued research and development in this domain will further refine these technologies, making them more reliable and cost-effective. As such systems gain wider adoption, they have the potential to become an integral part of transportation safety frameworks worldwide, paving the way for safer roads and more efficient traffic management.

1.2 ARTIFICIAL INTELLIGENCE

Artificial Intelligence (AI) is the simulation of human intelligence in machines that are programmed to think and act like humans. Learning, reasoning, problem-solving, perception, and language comprehension are all examples of cognitive abilities. Artificial intelligence is a method of making a computer, a computer-controlled robot, or a software think intelligently like the human mind. AI is accomplished by studying the patterns of the human brain and by analyzing the cognitive process. The outcome of these studies develops intelligent software and systems [6].

Artificial Intelligence (AI) is the theory and development of computer systems capable of performing tasks that historically require human intelligence, such as recognizing speech, making decisions, and identifying patterns. AI is an umbrella term that encompasses a wide variety of technologies, including machine learning, deep learning, and natural language processing (NLP).

Although the term is commonly used to describe a range of different technologies in use today, many disagree on whether these constitute artificial intelligence. Instead, some argue that much of the technology used in the real world today actually constitutes highly advanced machine learning that is simply a first step towards true artificial intelligence, or “general artificial intelligence” (GAI).

1.2.1 TYPES OF ARTIFICIAL INTELLIGENCE

Artificial Intelligence can be divided into two different categories: weak and strong.

- **WEAK AI**

Weak Artificial Intelligence embodies a system designed to carry out one job. Weak AI systems include video games such as the chess example from above and personal assistants such as Amazon's Alexa and Apple's Siri. You ask the assistant a question, and it answers it for you.[10].

- **STRONG AI**

Strong Artificial Intelligence systems are systems that carry on tasks considered to be human-like. These tend to be more complex and complicated systems. They are programmed to handle situations in which they may be required to problem solve without having a person intervene. These kinds of systems can be found in applications like self-driving cars or in hospital operating rooms [10].

1.3 DEEP LEARNING

According to Geoffrey Hinton (considered as the father of Deep Learning), deep learning (DL) allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction. Deep learning is a type of technology that allows computers to simulate how our brains work. It is a subset of machine learning, a branch of artificial intelligence [3]. In deep learning, a computer model learns to perform classification tasks directly from images, text, or sound. Deep learning models can achieve state-of-the-art accuracy, sometimes exceeding human-level performance. Models are trained by using a large set of labelled data and neural network architectures that contain many layers. It is a field that is based on learning and improving its own by examining computer algorithms.

Deep learning uses artificial neural networks that consist of multiple layers of nodes, also known as neurons, that can learn from large amounts of data. Deep learning can process unstructured data, such as images and text, and automatically extract important features and patterns from the data [3]. Deep learning can also perform different types of learning, such as supervised, unsupervised, and reinforcement learning. Deep learning is the technology behind many popular AI applications, such as chatbots, virtual assistants, self-driving cars, and more.

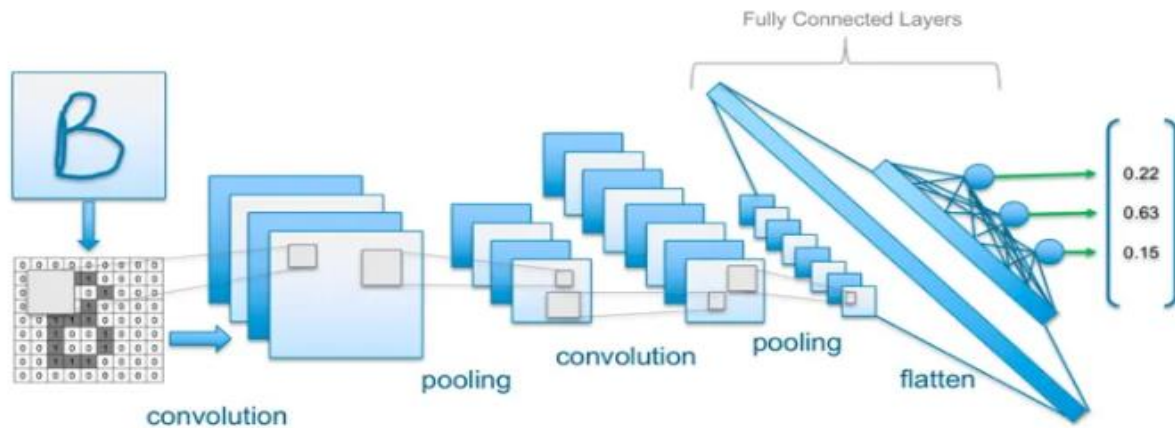


Figure 1.2. Deep Learning Architecture

1.4 DEEP LEARNING VS MACHINE LEARNING

Machine Learning and Deep Learning are revolutionary fields in the computer and information sciences area and are used widely in business applications. Machine Learning is an approach to train computers or machines to learn from past data or behavior so it can determine future data or behavior.

Deep Learning is a branch of Machine Learning where the Artificial Neural Network techniques and algorithms are used to train and learn. Due to the large volume of data points generated daily, there is a great need for the proper use of data and data management using the major techniques [11].

Another key difference is deep learning algorithms scale with data, whereas shallow learning converges. A key advantage of deep learning networks is that they often continue to improve as the size of your data increases. The difference in working of machines and deep learning is shown in figure 1.3.

In machine learning, a programmer must intervene directly in the action for the model to come to a conclusion. In the case of a deep learning model, the feature extraction step is completely unnecessary. The model would recognize these unique characteristics of a car and make correct predictions. Deep Learning models tend to increase their accuracy with the increasing amount of training data, where's traditional machine learning models such as SVM and Naive Bayes classifier stop improving after a saturation point [11].

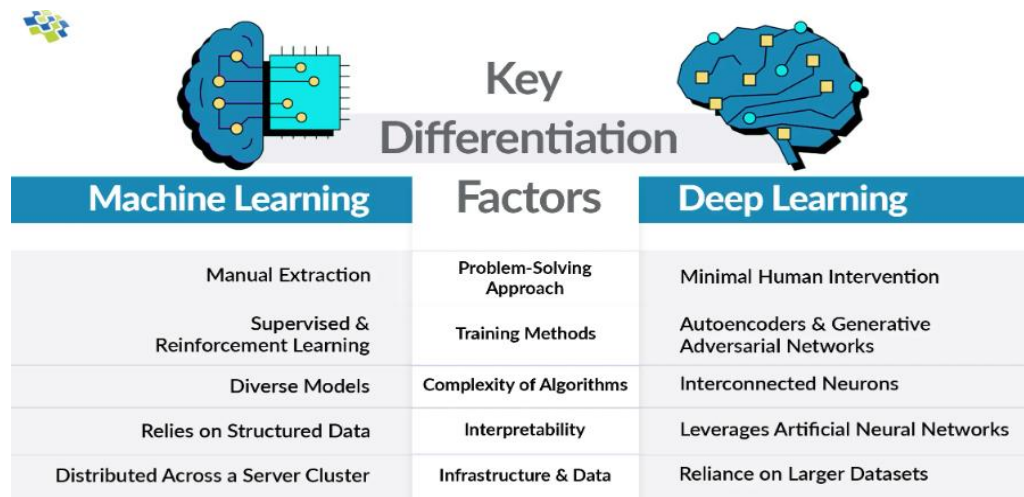


Figure 1.3. Machine Learning Vs Deep Learning

1.5 NEURAL NETWORKS

Deep learning algorithms attempt to draw similar conclusions as humans would by continually analyzing data with a given logical structure. To achieve this, deep learning uses a multi-layered structure of algorithms called neural networks. The design of the neural network is based on the structure of the human brain. Just as we use our brains to identify patterns and classify different types of information, neural networks can be taught to perform the same tasks on data [4].

Neural networks, also known as artificial neural networks (ANNs) or artificially generated neural networks (SNNs) are a subset of machine learning that provide the foundation of deep learning techniques. Their name and form are inspired by the human brain, and they replicate the way real neurons communicate with one another. Artificial neural networks (ANNs) are massively parallel systems comprised of a huge number of interconnected basic processors [4]. The Neural network architecture is shown in figure 1.4.

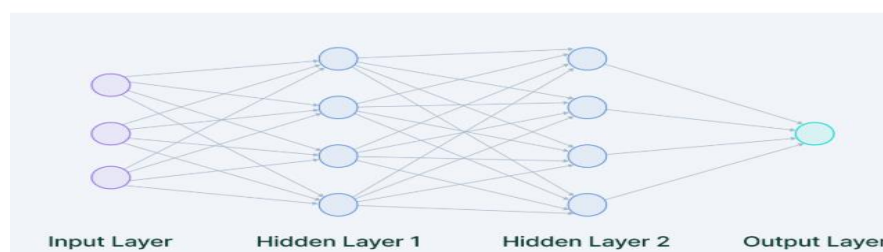


Figure 1.4. Neural Network Architecture

The individual layers of neural networks can also be thought of as a sort of filter that works from gross to subtle, increasing the likelihood of detecting and outputting a correct result. The human brain works similarly. Whenever we receive new information, the brain tries to compare it with known objects. The same concept is also used by deep neural networks.

With neural networks, we can group or sort unlabeled data according to similarities among the samples in this data. Or in the case of classification, we can train the network on a labeled data set to classify the samples in this data set into different categories [12].

1.6 YOLO V12

YOLOv12 (You Only Look Once version 12) is an advanced real-time object detection algorithm designed to achieve high accuracy and speed in computer vision tasks. It builds upon the evolution of earlier YOLO models by introducing improved architectural components, optimized feature extraction, and enhanced detection capabilities.

YOLOv12 follows a single-stage detection approach, where the entire image is processed in one pass through a deep neural network, making it significantly faster than traditional two-stage detectors. The model divides an input image into a grid and predicts bounding boxes, class probabilities, and confidence scores simultaneously.

One of the key improvements in YOLOv12 is the use of a more efficient backbone network for feature extraction, which helps capture both fine-grained and high-level semantic information. Additionally, it incorporates advanced feature pyramid networks (FPN) or path aggregation techniques to improve detection of objects at multiple scales, ensuring better performance for small and distant objects.[22]

YOLOv12 also enhances localization accuracy through improved bounding box regression and loss functions, reducing errors in object positioning. Another important feature is its lightweight design, which allows deployment on edge devices and real-time systems such as traffic surveillance and autonomous monitoring.

The algorithm is robust under varying environmental conditions, including low light, occlusion, and motion blur, making it suitable for real-world applications. Furthermore, YOLOv12 supports multi-class detection, enabling simultaneous identification of multiple objects within a single frame, such as vehicles, riders, helmets, and mobile phones. Its

optimized training process and use of large-scale datasets contribute to improved generalization and reduced overfitting.

1.6.1 ARCHITECTURE DIAGRAM OF YOLO V12

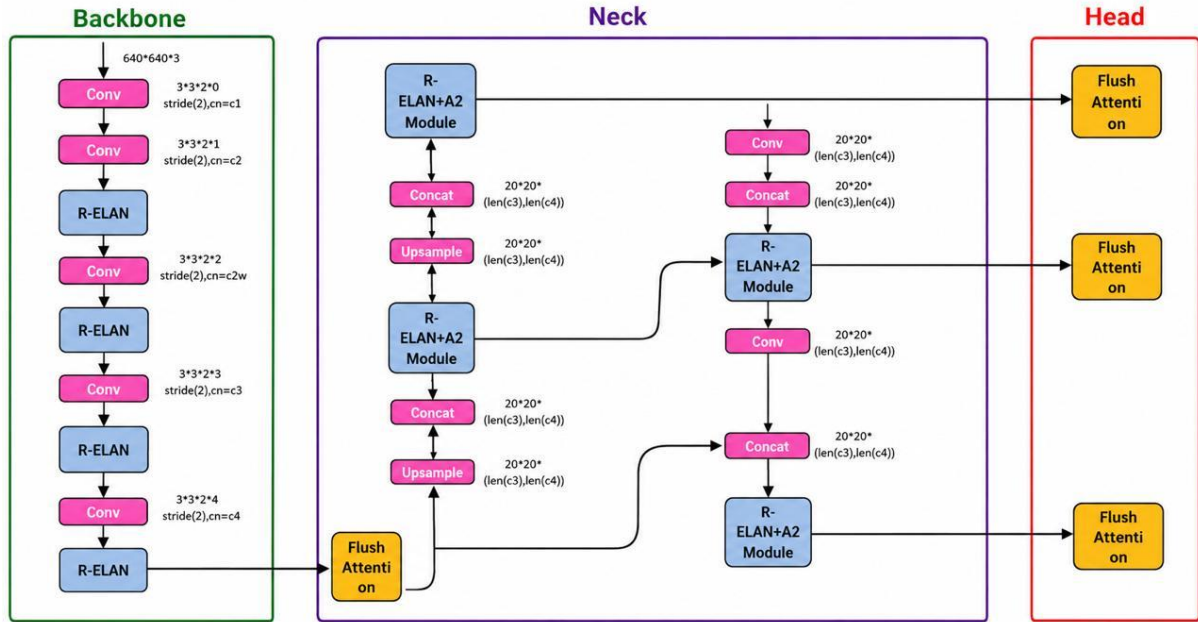


Figure 1.6.1. Architecture Diagram Of Yolo V12

The architecture of YOLOv12 follows a streamlined and efficient design consisting of three main components: the backbone, the neck, and the head. The backbone is responsible for feature extraction, where the input image is passed through multiple convolutional layers to capture both low-level features such as edges and textures and high-level semantic information like object shapes.[10]

YOLOv12 typically employs an optimized and lightweight backbone to improve speed while maintaining strong feature representation.

The extracted features are then passed to the neck, which enhances feature fusion using techniques such as Feature Pyramid Networks (FPN) or Path Aggregation Networks (PAN). This stage combines features from different scales, allowing the model to detect objects of varying sizes, including small and distant ones.

CHAPTER II

LITERATURE SURVEY

Title 1 : Automated detection of bicycle helmets using deep learning

Author : Felix Wilhelm Siebert, Christoffer Riis, Kira, Oguzhan Gül, Frederik Boe Huttle

Year: 2025

Description:

Bicycle helmets are a vital factor in reducing the severity of injuries during traffic accidents, making helmet usage an important metric in transport safety research. Traditionally, helmet-use data is collected through manual observation, which is expensive, time-consuming, and limited in scale.

To address these challenges, the study proposes an automated computer vision-based approach for detecting bicycle helmet usage from traffic video footage. Video data was collected from two observation sites in Copenhagen, Denmark, and a dataset consisting of 4,000 cyclists was manually annotated to indicate helmet use. A state-of-the-art object detection model was trained to identify cyclists and classify whether they were wearing helmets.

The developed model demonstrated strong performance in detecting active cyclists and estimating helmet usage. On the test dataset, helmet usage was only slightly underestimated by 0.52%, showing high accuracy and reliability.

Conclusion:

Helmets provide significant protection against maxillofacial injuries. However, higher-quality and more recent studies, particularly addressing e-bikes and e-scooters, are needed to strengthen evidence and guide policy.

Title 2 : Automatic Helmet Violation Detection Using Deep Learning Algorithms

Author : Md. Ameer Raza, J. Bhavyasri Tanmaya Moukthika

Year: 2025

Description:

Automated detection of motorcycle helmet usage from surveillance videos plays a critical role in strengthening road safety enforcement and awareness programs.

However, many existing approaches face notable limitations, such as the inability to reliably track individual motorcycles across consecutive video frames and difficulty in distinguishing between drivers and passengers when evaluating helmet compliance.

Additionally, earlier studies often depend on limited or homogeneous datasets, reducing robustness under varying traffic densities and real-world conditions.

To overcome these challenges, the proposed study introduces a convolutional neural network (CNN)–based multi-task learning (MTL) framework that simultaneously performs motorcycle detection, tracking, rider identification, and helmet usage classification.

By jointly learning visual similarity features for tracking and helmet classification within a single unified model, the system effectively links helmet status to specific riders over time.

Conclusion:

The study validates the feasibility of using lightweight deep learning models for helmet violation detection in real-world traffic conditions. It emphasizes scalability and efficiency, making the system suitable for real-time deployment in smart cities.

Title 3 : Real-Time Deep-Learning-Based Recognition of Helmet-Wearing Personnel on Construction Sites from a Distance

Author: Fatih Aslan and Yaşar Becerril

Year: 2025

Description:

Wearing personal protective equipment (PPE), especially helmets, is essential on construction sites to prevent severe head injuries and to comply with safety regulations.

Helmets are particularly important because they are highly visible in surveillance footage, making them suitable for automated worker detection and monitoring.

However, manual inspection of helmet usage and worker identity is inefficient, labour-intensive, and impractical for large or busy construction sites.

Both models achieved high precision and recall, with YOLOv5 showing slightly better performance for real-time symbol identification. The system was effective at identifying workers from distances of up to 10 meters. Additionally, a left-to-right, location-based symbol-ordering algorithm ensured consistent and reliable interpretation of symbols during inference.

Conclusion:

The two-stage pipeline demonstrates robust performance for helmet violation detection and license plate recognition.

Its strong accuracy and practical orientation highlight its suitability for deployment in Indian smart-city surveillance networks.

Title 4: Automatic Helmet Violation Detection Using Deep Learning Techniques

Author: Md. Ameer Raza, J. Bhavyasri Tanmaya Moukthika Dr. Varsha M, Niveditha MG,
Priyadarshini P, Punyashree B S, Sahana M K

Year: 2025

Description:

Automated helmet detection systems play a crucial role in improving road safety in India, where two-wheelers are the most commonly used mode of transportation.

A significant number of road accidents and fatalities occur due to helmet non-compliance, even though traffic safety laws are clearly defined.

To overcome limitations in manual enforcement, automated helmet violation detection systems based on CCTV surveillance and machine learning have been developed.

These systems use deep learning–based object detection techniques and are enhanced with Class Activation Mapping (CAM) to improve detection performance, particularly under low-light conditions.

The proposed approach achieves an accuracy of 85.3%, making it suitable for large-scale deployment in urban traffic environments.

Conclusion:

The proposed system demonstrates the integration of helmet and triple-ride detection into single surveillance framework.

Its strength lies in practical feasibility and low-cost deployment, though further validation with larger datasets is needed.

Title 5: HELMET VIOLATION DETECTION USING DEEP LEARNING

Author: Sri Uthra V, Sariga Devi V, Vaishali K S, Padma Priya S

Year: 2025

Description:

Two-wheelers are among the most used modes of transportation, particularly in densely populated regions, but they are also associated with a high rate of road accidents. A major contributor to fatal injuries is helmet non-compliance, as head injuries account for a large proportion of two-wheeler deaths.

To address this safety concern, the proposed system introduces an integrated and automated framework for detecting helmet rule violations and identifying offenders, thereby reducing reliance on manual traffic monitoring. The system combines multiple computer vision and machine learning techniques into a single pipeline.

First, motorcycles are detected and classified from traffic surveillance footage using a Support Vector Machine (SVM)–based vehicle classification approach.

Helmet detection is then performed using Convolutional Neural Networks (CNNs) to extract visual features from rider images, followed by an SVM classifier to determine the presence or absence of a helmet.

Conclusion:

The developed model effectively combines helmet violation and triple-riding detection within a unified intelligent monitoring system.

The system achieves reliable performance in real-time traffic environments while maintaining cost efficiency and scalability.

2.6 LITERATURE SURVEY TABLE:

TITLE	TECHNIQUE USED	CONCLUSION
AI-Based Helmet Violation detection for Traffic Management System	Optimized PerspectiveNet with EfficientNet v2 backbone, deep learning-based real-time helmet violation detection.	We conclude that architecture optimization enables accurate real-time traffic monitoring, guiding our project toward efficient and scalable safety applications.
Computer-vision based automatic rider helmet violation detection and vehicle identification in Indian smart city scenarios using NVIDIA TAO toolkit and YOLOv8	Two-step approach: DetectNet_v2 (ResNet18 + TAO) for rider detection, YOLOv8 for helmet + number plate detection, OCR for plate recognition; Custom dataset of 1,715 images	We conclude that combining multi-model deep learning improves reliability. This supports our project vision to use a multi-model approach for accurate detection, even in complex real-time traffic scenarios.
Automated detection of bicycle helmets using deep learning	Computer vision + state-of-the-art object detection algorithms trained on annotated cyclist datasets, real-world traffic video analysis.	We conclude that helmet detection can scale beyond motorbikes to bicycles, proving our project can extend to multiple road user categories for broader safety enforcement.
Smart Road Surveillance: Helmet Detection and Triple Ride Detection Using CNN and YOLO	YOLOv8s for motorcycle, rider, and plate detection; CNN for helmet/no-helmet classification; OCR (Tesseract) for license plate recognition; real-time triple ride detection	We conclude that integrating multiple detection tasks (helmet, license plate, triple ride) into a unified system enhances road safety enforcement.
An AI multitier system with lightweight classifier for automated helmetless biker detection	Custom CNN model BikeNet-12, datasets with helmet/no-helmet/safe-unsafe helmet classes, YOLOv8 for object detection in varied conditions.	We conclude that lightweight models can deliver high accuracy with less computation, highlighting efficiency as a key factor for our system's real-world deployment.

CHAPTER III

SYSTEM STUDY

3.1 EXISTING SYSTEM

In the existing system, helmet violation detection is largely dependent on manual monitoring by traffic police or on basic image processing techniques. Traffic officers are required to continuously observe CCTV footage or roadside camera feeds and visually identify riders who are not wearing helmets.

This approach is highly time-consuming and demands significant human effort, making it inefficient for monitoring large traffic volumes. Moreover, manual observation is prone to human error, fatigue, and inconsistency, which can lead to missed violations or incorrect enforcement actions. Some conventional systems attempt to automate helmet detection using simple image processing methods based on color, shape, or edge detection.

While these techniques offer partial automation, they lack robustness and perform poorly in real-world conditions. Variations in helmet color and design, changes in lighting conditions such as night-time or shadows, and complex or cluttered backgrounds significantly reduce their accuracy.

These systems also struggle with occlusions, camera angle variations, and low-resolution video feeds, making them unreliable for practical deployment. Overall, existing helmet violation detection systems lack full automation, real-time processing capability, and high accuracy.

Their dependence on manual intervention and fragile image processing rules limits scalability and effectiveness. These limitations clearly highlight the need for an advanced deep learning-based approach that can automatically learn discriminative features, operate reliably under diverse conditions, and provide efficient, real-time helmet violation detection for modern traffic management systems.

3.1.1 LIMITATION OF THE EXISTING SYSTEM

In the existing system, helmet violation detection is primarily carried out through manual monitoring by traffic police or by using basic image processing techniques. Traffic personnel observe CCTV footage or roadside camera feeds and visually identify riders who are not wearing helmets.

This method is highly time-consuming, labor-intensive, and prone to human errors caused by fatigue, distraction, or subjective judgment. To reduce manual effort, some traditional systems apply simple image processing approaches based on predefined color, shape, or edge features to detect helmets.

However, these techniques lack robustness and often fail in real-world conditions such as poor lighting, night-time scenarios, varying helmet designs, camera angle changes, and complex backgrounds. Additionally, they are unable to handle occlusions or crowded traffic environments effectively. As a result, existing systems lack automation, real-time processing capability, and reliable accuracy.

These limitations restrict scalability and make them unsuitable for modern traffic surveillance needs, emphasizing the necessity for more advanced and intelligent detection approaches.

3.2 PROPOSED SYSTEM

The proposed system overcomes the limitations of existing helmet violation detection methods by adopting a fully automated deep learning–based framework built on YOLOv12, which offers improved accuracy, faster inference, and better generalization across real-world traffic conditions. YOLOv12 is utilized to precisely detect helmet usage by learning complex visual features, enabling reliable differentiation between helmeted and non-helmeted riders even in challenging scenarios such as varying lighting conditions, different helmet styles, dense traffic, and diverse camera angles.

The system is trained using robust and diverse datasets to ensure consistent performance in both urban roads and highways. Upload video processed in real time, allowing instant identification of helmet violations without the need for continuous human supervision. When a rider without a helmet is detected, the system automatically triggers alert

mechanisms by notifying authorized institutions such as traffic police departments and issuing warning notifications to riders.

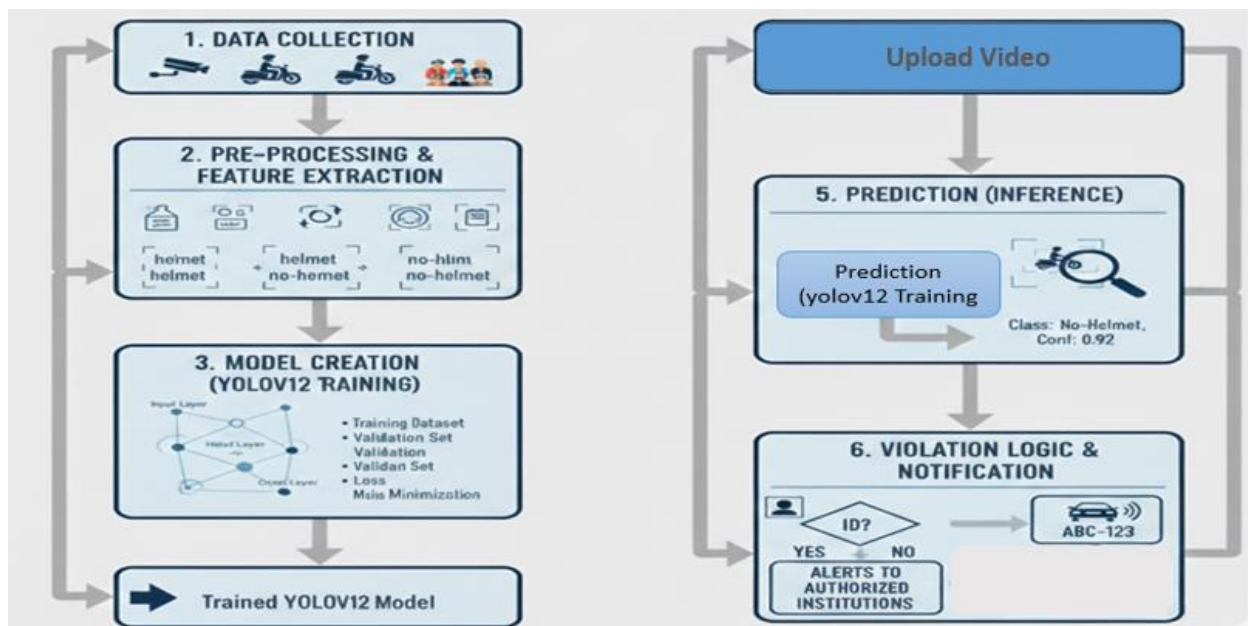
This immediate response supports proactive enforcement and increases rider awareness, encouraging compliance with helmet regulations. The automated workflow significantly reduces human error, monitoring effort, and enforcement delays, making the system suitable for large-scale and continuous deployment.

By combining high-accuracy detection, real-time processing, and automated alert generation, the proposed system ensures efficient enforcement and practical implementation. Overall, the system enhances traffic monitoring capabilities, strengthens helmet law enforcement, and contributes to reducing accident-related injuries and fatalities, thereby improving overall road safety.

3.2.1 ADVANTAGE OF THE PROPOSED SYSTEM:

- Provides highly accurate helmet detection using advanced deep learning techniques, ensuring reliable performance across diverse real-world traffic conditions.
- Enables real-time processing video, allowing instant identification of helmet violations without manual intervention.
- Automatically generates alerts and warning notifications, improving enforcement efficiency and encouraging rider compliance.
- Reduces human effort, operational cost, and errors associated with manual monitoring by traffic authorities.
- Scalable and practical for large-scale deployment in urban and highway traffic surveillance systems to enhance overall road safety.

3.2.3 ARCHITECTURE DIAGRAM:



The architecture diagram illustrates a two-phase framework consisting of an offline training phase and an online real-time deployment phase for helmet violation detection. In the offline training phase, the process begins with data collection, where images and videos of motorcyclists under different conditions are gathered.

These data are then passed through a pre-processing and feature extraction stage, where relevant features such as helmet and no-helmet regions are prepared and annotated. Next, model creation is carried out using YOLOv12 training, where the dataset is divided into training and validation sets, and the model learns to minimize detection loss through iterative optimization.

This phase results in a trained YOLOv12 model capable of accurately distinguishing helmeted and non-helmeted riders. In the online real-time deployment phase, upload video inputs are fed into the trained model for prediction (inference). The system classifies each detected rider in real time with confidence scores. When a no-helmet violation is detected, violation logic is applied to verify identification details, and the system automatically generates alerts to authorized institutions and sends warning notifications to riders. This integrated architecture ensures continuous monitoring, real-time enforcement, and efficient road safety management.

CHAPTER IV

SYSTEM REQUIREMENTS

4.1 SOFTWARE REQUIREMENTS

- OS: Windows 10 (or above)
- LANGUAGE: Python 3.11
- Operating System: Windows 10 and above
- Coding language: Python 3.10.2 and above
- Visual studio
- Front End: HTML, CSS, JavaScript
- Back End: SQL lite
- Tensor flow

4.2 HARDWARE REQUIREMENTS

- **PROCESSOR:** Intel(R) core (TM) i5-10thGEN (or above)
- **RAM:** 4 GB (or above)
- **PROCESSOR SPEED:** 2GHz
- **HARD DISK:** 512 GB SSD (or above)
- **KEYBOARD:** Standard Windows Keyboard
- **MOUSE:** 2 or 3 button mouse

4.2.1 PYTHON:

Python is an interpreted, high-level, general-purpose programming language. Created by Guido van Rossum and first released in 1991, Python's design philosophy emphasizes code readability with its notable use of significant whitespace. Its language constructs and object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects.

Python is dynamically typed and garbage is collected. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming. Python is oftendescribed as a "batteries included" language due to its comprehensive standard library.

Python is one of those rare languages which can claim to be both simple and powerful. You will find yourself pleasantly surprised to see how easy it is to concentrate on the solution to the problem rather than the syntax and structure of the language you are programming in. Python is an interpreted high-level and general-purpose programming language.

Python's design philosophy emphasizes code readability with its notable use of significant indentation. Its language constructs and object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects. Python is dynamically-typed and garbage-collected. It supports multiple programming, including structured, object-oriented, and functional programming.

Python is often described as a "batteries included" language due to its comprehensive standard library. It supports multiple programming including structured, object-oriented, and functional programming.

4.2.2 TENSORFLOW & KERAS:

TensorFlow is a free and open-source software library for machine learning and artificial intelligence. It can be used across a range of tasks but has a particular focus on training and inference of deep neural networks. TensorFlow was developed by the Google Brain team for internal Google use in research and production. TensorFlow can be used in a wide variety of programming languages, including Python, JavaScript, C++, and Java. TensorFlow serves as the core platform and library for machine learning. TensorFlow's APIs use Keras to allow users to make their own machine learning models.

In addition to building and training their model, TensorFlow can also help load the data to train the model and deploy it using TensorFlow Serving. Auto Differentiation is the process of automatically calculating the gradient vector of a model with respect to each of its parameters. With this feature, TensorFlow can automatically compute the gradients for the parameters in a model, which is useful to algorithms such as backpropagation which require gradients to optimize performance.

Keras is an open-source software library that provides a Python interface for artificial neural networks. Keras acts as an interface for the TensorFlow library. Keras contains numerous implementations of commonly used neural-network building blocks such as layers, objectives, activation functions, optimizers, and a host of tools to make working with image and text data easier to simplify the coding necessary for writing deep neural network code.

The code is hosted on GitHub, and community support forums include the GitHub issues page, and a Slack channel. Keras has support for convolutional and recurrent neural networks. It supports other common utility layers like dropout, batch normalization, and pooling. Keras allows users to productize deep models on smartphones (iOS and Android), on the web, or on the Java Virtual Machine.

4.2.3 SQLITE3:

SQLite3 is a lightweight, open-source, and self-contained database management system that is widely used across software applications for managing structured data. Unlike traditional database systems such as MySQL or PostgreSQL, which operate as separate server processes, SQLite3 is embedded directly into the application, making it extremely fast, efficient, and easy to use. It stores the entire database, including tables, indexes, and the schema, in a single file on the disk, which simplifies data management and portability. Since it requires no setup or configuration, developers can integrate SQLite3 seamlessly into projects without the overhead of managing a server. SQLite3 supports most features of SQL, such as transactions, joins, triggers, and views, ensuring robust relational data management. One of its key strengths is its ability to handle moderate-sized databases with minimal resource consumption, making it highly suitable for mobile applications, desktop software, IoT devices, and embedded systems. For example, it is the default database engine used by Android, iOS, and many popular browsers like Chrome and Firefox. Additionally, SQLite3 is fully ACID-compliant (Atomicity, Consistency, Isolation, Durability), which ensures reliability even in cases of system crashes or power failures. Its small footprint and zero-administration design make it ideal for scenarios where simplicity, stability, and low maintenance are critical. However, it is not intended for large-scale, high-concurrency enterprise applications where distributed servers and heavy parallel transactions are required. Instead, it excels in use cases like application data storage, prototyping, testing, and systems with limited hardware resources. Overall, SQLite3 offers a balance of simplicity, reliability, and functionality, making it one of the most widely used and trusted embedded database engines in the world today.

4.2.4 FLASK:

Flask is a lightweight, open-source web framework written in Python that is widely used for developing web applications and RESTful APIs. Designed with simplicity and flexibility in mind, Flask follows the microframework philosophy, meaning it provides the

essential tools needed to build web applications without imposing dependencies or a rigid project structure. This makes it highly adaptable for both small-scale projects and large, complex applications.

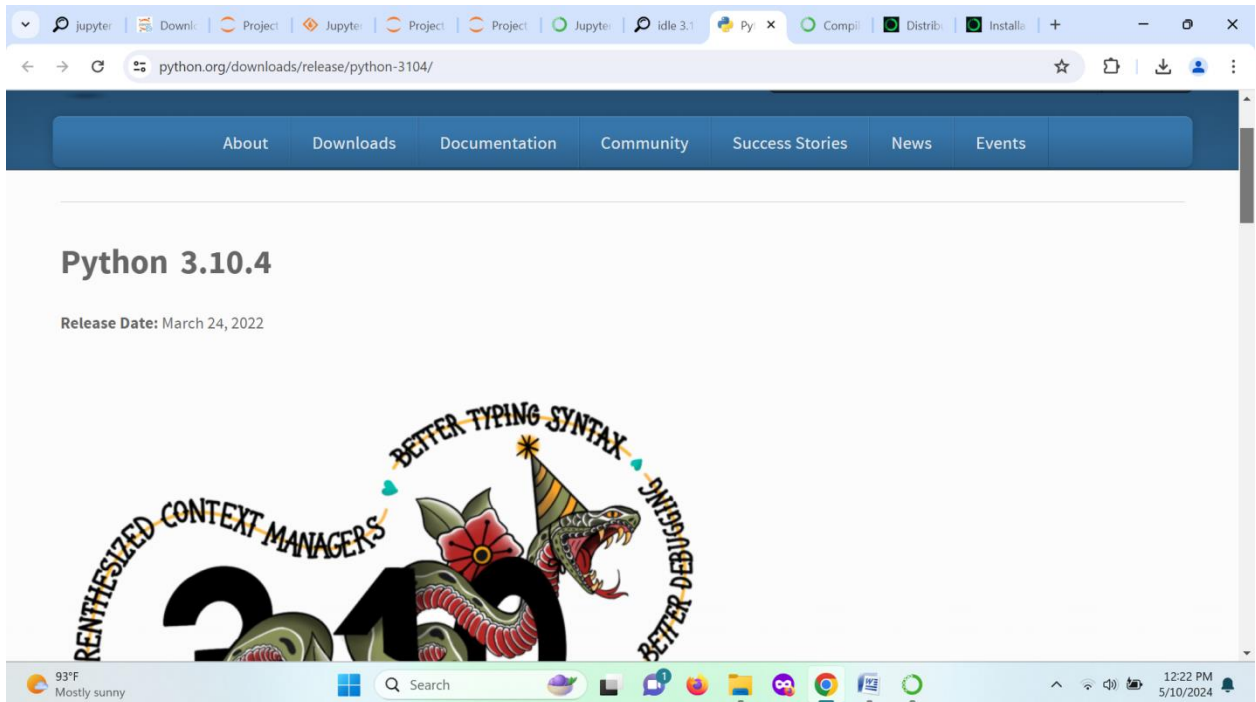
At its core, Flask includes a built-in development server, URL routing, and a templating engine called Jinja2, which allows developers to dynamically generate HTML pages. It also supports request handling, session management, and error handling, making it a complete yet minimalistic framework. One of Flask's key advantages is its modularity: developers can easily extend its functionality using third-party libraries or integrate it with various databases such as SQLite, MySQL, or PostgreSQL. This modular approach allows for rapid prototyping while maintaining the flexibility to scale applications as requirements grow.

Flask is particularly popular for developing RESTful APIs because it provides straightforward request and response handling, JSON support, and the ability to define clear endpoints. Additionally, Flask's simplicity and minimalism make it an excellent learning platform for beginners in web development, as it emphasizes understanding the fundamentals without abstracting too much functionality. Despite its lightweight design, Flask is production-ready and can handle complex applications when combined with extensions for authentication, form validation, caching, and more.

Its active community ensures continuous development, support, and availability of numerous plugins and extensions, making it a robust choice for web development projects. Overall, Flask strikes a balance between simplicity and extensibility, offering developers the freedom to structure applications as they see fit while providing essential tools for building modern, scalable, and maintainable web applications.

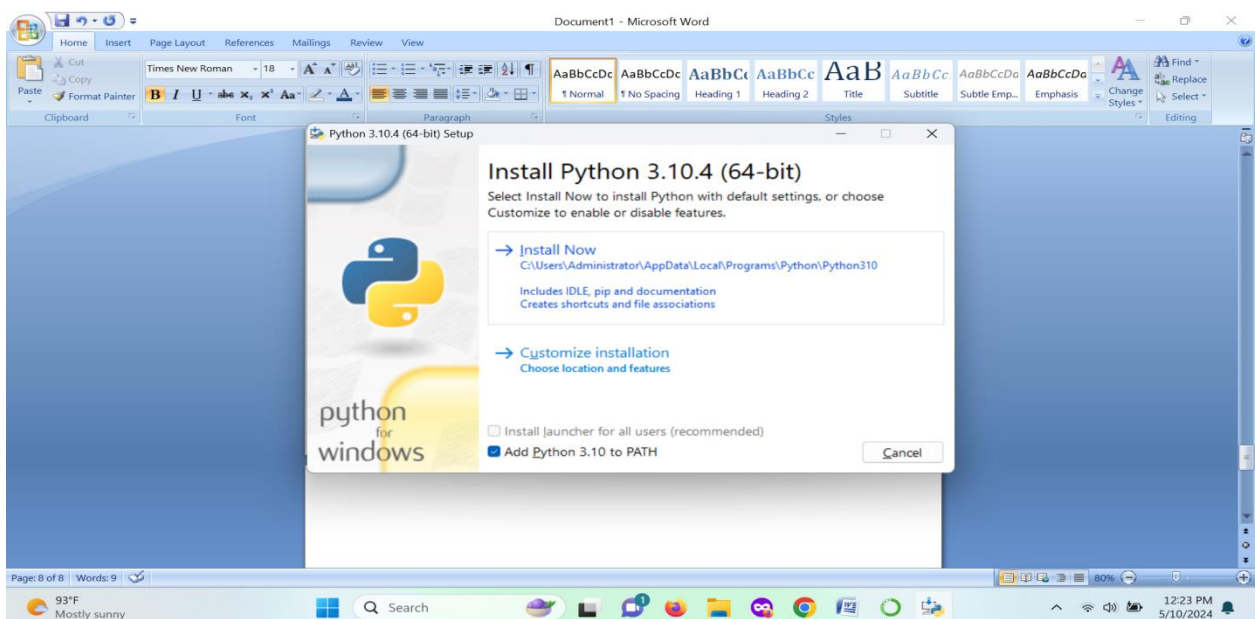
4.3 ENVIRONMENTAL SETUP:

4.3.1 DOWNLOAD IDLE PYTHON:



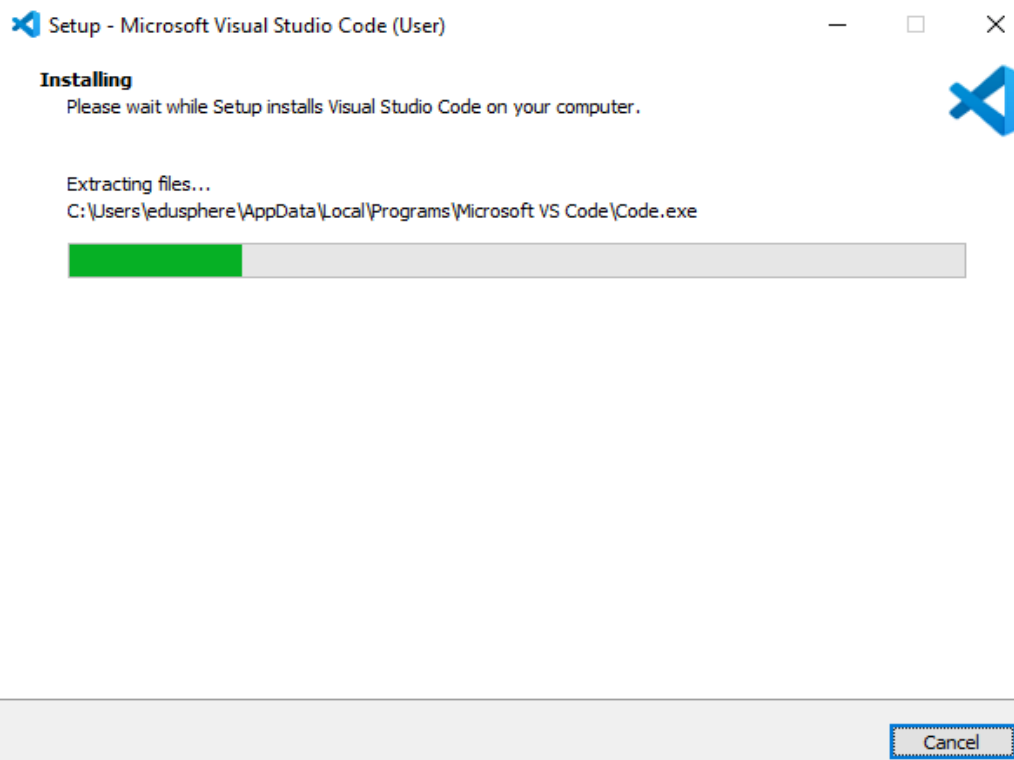
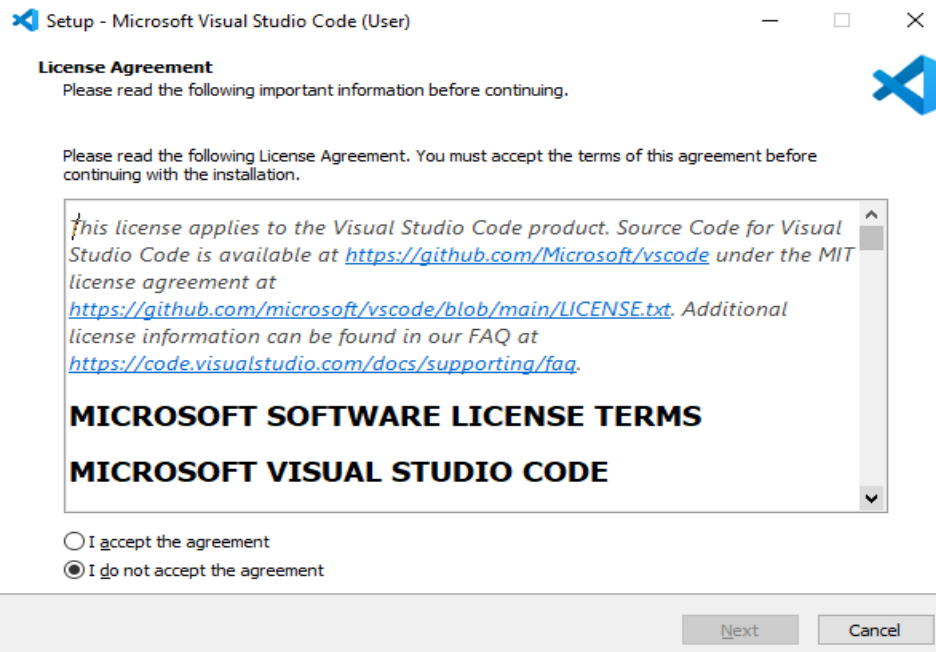
Download the python idle 3.10.4 and link is given below.

<https://www.python.org/downloads/release/python-3104/>



Click to install and process the initialization.

4.3.2 VISUAL STUDIO CODE:



Completing the Visual Studio Code Setup Wizard

Setup has finished installing Visual Studio Code on your computer. The application may be launched by selecting the installed shortcuts.

Click Finish to exit Setup.

☒ Launch Visual Studio Code



Finish

4.3.3 NUMPY:

```
C:\Users\acer>pip install numpy
Collecting numpy
  Downloading https://files.pythonhosted.org/packages/ce/ad/2e88f36b56f64f70c081b32fa5512dacedf12005ccb0c2d300d44dcc1215/numpy-1.17.4-cp37-cp37m-win32.whl (10.7MB)
    | 10.7MB 467kB/s
Installing collected packages: numpy
Successfully installed numpy-1.17.4
```

Pip Install NumPy.

4.3.4 TENSOR FLOW:

```
(tensor_env) kumar_satyam@satyam:~/gfg/tensor$ pip install --upgrade tensorflow
Collecting tensorflow
  Using cached tensorflow-2.5.0-cp36-cp36m-manylinux2010_x86_64.whl (454.3 MB)
Collecting numpy~=1.19.2
  Using cached numpy-1.19.5-cp36-cp36m-manylinux2010_x86_64.whl (14.8 MB)
Collecting wheel~=0.35
  Using cached wheel-0.36.2-py2.py3-none-any.whl (35 kB)
Collecting keras-nightly~=2.5.0.dev
  Using cached keras_nightly-2.5.0.dev2021032900-py2.py3-none-any.whl (1.2 MB)
Collecting h5py~=3.1.0
  Using cached h5py-3.1.0-cp36-cp36m-manylinux1_x86_64.whl (4.0 MB)
Collecting six~=1.15.0
  Using cached six-1.15.0-py2.py3-none-any.whl (10 kB)
Collecting grpcio~=1.34.0
  Using cached grpcio-1.34.1-cp36-cp36m-manylinux2014_x86_64.whl (4.0 MB)
```

Pip Install TensorFlow.

4.3.5 PANDAS:

```
Microsoft Windows [Version 10.0.19043.1083]
(c) Microsoft Corporation. All rights reserved.

C:\Users\>pip install pandas
Collecting pandas
  Downloading https://files.pythonhosted.org/packages/cb/e3/c0bc0f1b3835564f69094135de105a3def2eeb2689338a906bfc659c99d0/pandas-1.3.0-cp38-cp38-win_amd64.whl (10.2MB)
    | 10.2MB 3.3MB/s
Collecting pytz>=2017.3 (from pandas)
  Downloading https://files.pythonhosted.org/packages/70/94/784178ca5dd892a98f113cdd923372024dc04b8d40abe77ca76b5fb90ca6/pytz-2021.1-py2.py3-none-any.whl (510kB)
    | 512kB 6.4MB/s
Collecting python-dateutil>=2.7.3 (from pandas)
  Downloading https://files.pythonhosted.org/packages/36/7a/87837f39d0296e723bb9b62bbb257d0355c7f6128853c78955f57342a56d/python_dateutil-2.8.2-py2.py3-none-any.whl (247kB)
    | 256kB ...
Collecting numpy>=1.17.3 (from pandas)
  Downloading https://files.pythonhosted.org/packages/df/22/b74e5cedee1e3f108c986bd0b75600997d8b25def334a68f08d372db523/numpy-1.21.0-cp38-cp38-win_amd64.whl (14.0MB)
    | 14.0MB 2.2MB/s
Collecting six>=1.5 (from python-dateutil>=2.7.3->pandas)
  Downloading https://files.pythonhosted.org/packages/d9/5a/e7c31adbe875f2abbb91bd84cf2dc52d792b5a01506781dbcf25c91daf11/six-1.16.0-py2.py3-none-any.whl
Installing collected packages: pytz, six, python-dateutil, numpy, pandas
Successfully installed numpy-1.21.0 pandas-1.3.0 python-dateutil-2.8.2 pytz-2021.1 six-1.16.0
WARNING: You are using pip version 19.2.3, however version 21.1.3 is available.
You should consider upgrading via the 'python -m pip install --upgrade pip' command.
```

Pip Install Pandas.

4.3.6 SCIKIT-LEARN:

```
C:\Users\Geeks>pip install scikit-learn
Collecting scikit-learn
  Downloading scikit_learn-0.24.2-cp38-cp38-win_amd64.whl (6.9 MB)
    | 6.9 MB 6.8 MB/s
Requirement already satisfied: joblib>=0.11 in c:\users\geeks\anaconda3\lib\site-packages (from scikit-learn) (1.0.1)
Requirement already satisfied: threadpoolctl>=2.0.0 in c:\users\geeks\anaconda3\lib\site-packages (from scikit-learn) (2.2.0)
Requirement already satisfied: scipy>=0.19.1 in c:\users\geeks\anaconda3\lib\site-packages (from scikit-learn) (1.7.1)
Requirement already satisfied: numpy>=1.13.3 in c:\users\geeks\anaconda3\lib\site-packages (from scikit-learn) (1.20.3)
Installing collected packages: scikit-learn
Successfully installed scikit-learn-0.24.2
```

Pip Install Scikit-Learn.

CHAPTER V

IMPLEMENTATION DETAILS

The implementation of the proposed helmet detection system using YOLOv12 involves a systematic pipeline that integrates data handling, model training, and real-time deployment. Initially, the collected dataset is organized into training, validation, and testing directories, along with corresponding annotation files in YOLO format. The implementation is typically carried out using Python with deep learning frameworks such as PyTorch or TensorFlow, along with libraries like OpenCV for image processing. The YOLOv12 model is configured by defining class labels (helmet and no helmet), input image size, and training parameters. Pretrained weights are often utilized to accelerate convergence and improve detection accuracy through transfer learning. During training, the model processes batches of images, learns feature representations, and updates its parameters using optimization algorithms like stochastic gradient descent or Adam. Performance is monitored using validation data to prevent overfitting and ensure generalization. Once training is complete, the model is evaluated on test data using metrics such as precision, recall, and mean Average Precision (mAP). For deployment, the trained model is integrated into a real-time detection system where video streams from surveillance cameras are captured using OpenCV. Each frame is passed through the YOLOv12 model, which detects riders and classifies helmet usage by generating bounding boxes and confidence scores. Non-Maximum Suppression is applied to refine detections and remove overlaps. The system can further be connected to an alert mechanism that generates notifications when violations are detected.

5.1 Data Collection

Data collection is the foundational step in developing an effective deep learning model for traffic violation detection. In this system, datasets are obtained from open-source platforms such as Kaggle, which provides a wide variety of annotated images and video datasets related to traffic surveillance. These datasets typically include images of motorcyclists, vehicles, and road scenes under different environmental conditions. For this project, relevant datasets are selected that contain instances of helmet usage and helmetless riders. The availability of labeled data, such as bounding box annotations and class labels, is crucial for supervised learning tasks like object detection. Kaggle datasets often come in formats like COCO or Pascal VOC, which are compatible with modern object detection

frameworks. Additionally, data diversity is ensured by including images captured under different lighting conditions, weather scenarios (rain, fog, night), and camera angles to improve the model's generalization ability. In some cases, multiple datasets may be combined to create a more comprehensive training set. Data augmentation techniques such as flipping, rotation, and scaling can also be applied at this stage to artificially increase dataset size and variability. Proper data collection ensures that the model learns from real-world scenarios, ultimately improving its robustness and accuracy in detecting helmet violations in practical environments.

5.2 pre-processing

Pre-processing is a critical step that prepares the collected data for efficient and accurate model training. Raw datasets obtained from Kaggle may contain inconsistencies such as varying image sizes, noise, irrelevant objects, or incomplete annotations. Therefore, preprocessing involves cleaning and standardizing the data to ensure uniformity. First, images are resized to a fixed resolution compatible with the YOLOv12 model, enabling faster computation and consistent input dimensions. Noise reduction techniques and normalization are applied to enhance image quality and stabilize pixel value distributions. Annotation files are also verified and converted into the required format (such as YOLO format), where bounding box coordinates are normalized relative to image dimensions. Data augmentation techniques, including rotation, flipping, brightness adjustment, and cropping, are applied to improve model robustness and prevent overfitting. These transformations help the model learn variations in object orientation, lighting, and scale. Additionally, dataset splitting is performed during preprocessing to divide the data into training, validation, and testing sets. This ensures unbiased model evaluation and prevents data leakage. Proper preprocessing significantly enhances the learning capability of the model by reducing noise, improving data consistency, and increasing variability. As a result, it plays a crucial role in improving overall performance, accuracy, and generalization of the YOLOv12-based helmet detection system.

5.3 Feature Extraction

Feature extraction is the process of identifying and representing important visual patterns from input images that help in detecting and classifying objects. In the YOLOv12 framework, feature extraction is performed automatically using a deep convolutional neural network (CNN) backbone. The backbone processes input images through multiple

convolutional layers, each designed to capture specific levels of information. Early layers focus on low-level features such as edges, corners, and textures, while deeper layers capture high-level semantic features like object shapes and contextual relationships. This hierarchical feature of learning enables the model to understand complex patterns required for detecting helmet usage. YOLOv12 enhances feature extraction by using an optimized backbone that balances computational efficiency and accuracy. Additionally, feature maps generated at different stages are passed to the neck (such as FPN or PAN), which combines multi-scale features to improve detection of both small and large objects. This is particularly important in traffic scenarios where helmets may appear small or partially occluded. Effective feature extraction ensures that the model can distinguish between helmet and no-helmet classes. By learning meaningful and discriminative features, the system achieves improved detection of accuracy and robustness in diverse real-world conditions.

5.4 Model Creation Using YOLOv12

Model creation involves designing, configuring, and training the YOLOv12 object detection model for helmet violation detection. The YOLOv12 model is initialized with a predefined architecture consisting of backbone, neck, and detection of head components. During this stage, custom classes such as “helmet” and “no helmet” are defined based on the problem requirements. The preprocessed dataset is then fed into the model for training. Transfer learning can be applied by initializing the model with pretrained weights, which helps improve performance and reduce training time. The training process involves optimizing model parameters using techniques such as stochastic gradient descent (SGD) or Adam optimizer, along with appropriate loss functions that account for bounding box regression, classification, and object confidence. Hyperparameters such as learning rate, batch size, and number of epochs are carefully selected to ensure stable and efficient training. During training, the model learns to detect riders and classify helmet usage accurately by minimizing prediction errors. Validation is performed periodically to monitor performance and avoid overfitting. The lightweight nature of YOLOv12 allows for faster training and real-time inference capabilities.

5.5 Test Data

Test data is used to evaluate the performance and generalization capability of the trained YOLOv12 model. It consists of a separate subset of the dataset that is not used during

training or validation, ensuring an unbiased assessment of the model's effectiveness. The test dataset includes images representing real-world traffic scenarios with varying conditions such as different lighting, weather, and traffic density. Proper selection of test data is crucial, as it should reflect the diversity and complexity of practical environments. During testing, the trained model processes each image and generates predictions, including bounding boxes, class labels, and confidence scores. These predictions are then compared with ground truth annotations to evaluate performance metrics such as precision, recall, F1-score, and mean Average Precision (mAP). High precision indicates fewer false positives, while high recall indicates effective detection of actual violations. Testing also helps identify limitations of the model, such as difficulty in detecting small objects or performance degradation under poor lighting conditions. Visualization of detection results is often performed to qualitatively assess model behavior. By analyzing test results, further improvements can be made through hyperparameter tuning, data augmentation, or model optimization.

5.6 Prediction

Prediction is the final stage where the trained YOLOv12 model is deployed to detect helmet violations in real-time or from stored images and videos. In this phase, input data such as live camera feeds or recorded footage is passed through the trained model. The model processes each frame and outputs predictions in the form of bounding boxes, class labels, and confidence scores. These predictions indicate the presence of riders and whether they are wearing helmets or not. YOLOv12 performs prediction in a single forward pass, ensuring high-speed processing is suitable for real-time applications. Non-Maximum Suppression (NMS) is applied to eliminate duplicate detections and retain the most accurate bounding boxes. The system can also be integrated with alert mechanisms that trigger notifications or warnings when violations are detected. For example, alerts can be sent to traffic authorities or displayed as caution messages. Visualization techniques such as overlaying bounding boxes on video frames help in understanding detection results. The prediction stage demonstrates the practical utility of the system, enabling automated monitoring and enforcement of helmet safety rules.

CHAPTER VI

RESULT AND DISCUSSION

The results and discussion of the proposed YOLOv12-based helmet detection system demonstrate its effectiveness in accurately identifying helmet usage in real-world traffic scenarios. The model was evaluated using a separate test dataset consisting of diverse images captured under varying environmental conditions, including different lighting levels, weather situations, and traffic densities. The system achieved high performance in terms of precision, recall, and mean Average Precision (mAP), indicating its ability to correctly detect riders wearing helmets as well as those without helmets while minimizing false detections. The use of a deep learning-based single-stage detector like YOLOv12 enabled fast processing speeds, making the system suitable for real-time applications.

Experimental results show that the model performs well even when helmets are partially occluded or when riders appear at different distances and angles. Compared to traditional methods and earlier lightweight models, the proposed approach offers improved detection accuracy and better generalization due to enhanced feature extraction and multi-scale detection capabilities.

However, certain challenges were observed during testing, such as slight performance degradation in extremely low-light conditions or when image quality is poor. Despite these limitations, the system maintains consistent and reliable performance across most scenarios. The integration of real-time prediction and alert mechanisms further enhances its practical usability for traffic monitoring and enforcement.

6.1 INPUT LAYER

The Input Layer in the proposed YOLOv12-based helmet detection system plays a crucial role in preparing traffic surveillance images for efficient processing and accurate detection. In this system, input data is obtained from CCTV cameras or Kaggle datasets containing images of motorbike riders.

These images are first resized to a fixed resolution (such as 640×640) to ensure uniformity and compatibility with the YOLOv12 architecture. This resizing helps maintain consistent spatial dimensions, enabling faster batch processing and real-time performance.

After resizing, the pixel values of the images are normalized to a range between 0 and 1 to stabilize training and improve convergence of the model. The normalization process can be represented as:

$$x' = \frac{x}{255}$$

Where x is the original pixel value and x' is the normalized value. The input image is then structured into a tensor of dimensions H , times W , times C , where H and W represent height and width, and $C=3$ corresponds to RGB channels. Furthermore, the image is conceptually divided into a grid of size S , where each grid cell is responsible for detecting objects, whose centers fall within that region. This grid-based representation allows the model to perform detection in a single forward pass. In the context of the proposed system, the input layer ensures that diverse traffic images—captured under different lighting and environmental conditions—are standardized and efficiently processed, thereby enabling accurate detection of helmet usage in real-time traffic monitoring applications.

6.2 Convolutional Layers

The Convolutional Layers in the proposed YOLOv12-based helmet detection system are the core components responsible for extracting meaningful features from input images. After preprocessing in the input layer, the image is passed through multiple convolutional layers, where filters (kernels) slide over the image to detect important visual patterns such as edges, textures, shapes, and object boundaries. In early layers, simple features like edges and corners are captured, while deeper layers learn complex patterns such as the structure of helmets and rider shapes. Each convolutional layer applies multiple filters to generate feature maps, which highlight different aspects of the image. These layers are typically followed by activation functions (such as REL or Siu) and batch normalization to improve learning efficiency and stability. The convolution operation can be mathematically expressed as:

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) \cdot K(m, n)$$

Where X represents the input image, K is the convolution kernel (filter), and $Y(i, j)$ is the output feature map at position (i, j) . This operation enables the model to focus on relevant spatial features while reducing noise and redundant information. In the proposed system,

convolutional layers play a vital role in distinguishing between helmet and no-helmet classes by learning discriminative features from traffic images. Their ability to automatically extract hierarchical features significantly improves detection accuracy and robustness, making them essential for real-time helmet detection in diverse environmental conditions.

6.3 Batch Normalization Layers

The Batch Normalization Layers in the proposed YOLOv12-based helmet detection system play a crucial role in improving training stability, convergence speed, and overall model performance. During deep neural network training, the distribution of inputs to each layer can change as the model updates its weights, a phenomenon known as internal covariate shift. Batch normalization addresses this issue by normalizing the activations of each layer for every mini-batch, ensuring that the data fed into subsequent layers maintains a consistent distribution. This normalization allows the model to train faster and reduces sensitivity to initialization and learning rates. In the context of the proposed system, batch normalization is applied after convolutional layers to stabilize feature extraction and improve the detection of helmets under varying lighting and environmental conditions. It also acts as a regularizer, reducing the need for additional techniques like dropout and helping prevent overfitting. The batch normalization process can be mathematically expressed as:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

Where, x is the input activation, μ is the mean of the batch, σ^2 is the variance, and ϵ is a small constant added for numerical stability. The normalized output $\hat{x}$ is then scaled and shifted using learnable parameters. By maintaining stable input distributions across layers, batch normalization enhances the robustness and efficiency of the YOLOv12 model, leading to improved accuracy in helmet detection and better generalization to real-world traffic scenarios.

6.4 Accuracy:

Accuracy is a fundamental evaluation metric used to measure the overall performance of the proposed YOLOv12-based helmet detection system. It represents the proportion of correctly predicted instances out of the total number of predictions made by the model. In this system, accuracy reflects how effectively the model can classify riders as wearing a helmet or

not wearing a helmet. A high accuracy indicates that the model is able to correctly identify most of the instances, thereby demonstrating its reliability in real-world traffic monitoring scenarios. However, accuracy alone may not always provide a complete picture, especially in cases where the dataset is imbalanced, such as having more helmet-wearing instances than non-helmet cases. Despite this limitation, it remains an important baseline metric for evaluating model performance during testing. The mathematical representation of accuracy is given as:

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Where, TP (True Positives) represents correctly detected helmet cases, TN (True Negatives) represents correctly detected no-helmet cases, FP (False Positives) represents incorrect detections where the model predicts a helmet but none is present, and FN (False Negatives) represents missed detections where the model fails to identify the absence of a helmet. In the proposed system, achieving high accuracy ensures that the model can reliably distinguish between helmet and no-helmet conditions. This contributes to effective enforcement of safety regulations and enhances the practical deployment of the system in intelligent traffic management applications.

6.5 LOSS

The loss function in the proposed YOLOv12-based helmet detection system is a critical component used to measure the difference between the model's predictions and the actual ground truth values during training. It guides the learning process by indicating how well the model is performing and how its parameters should be updated to improve accuracy. In object detection tasks like helmet detection, the loss function is typically a combination of multiple components, including classification loss (for identifying helmet or no helmet), localization loss (for predicting accurate bounding box coordinates), and confidence loss (for determining whether an object is present). By minimizing total loss, the model learns to make more precise predictions over time. A commonly used representation of total loss in YOLO-based models is:

$$Loss = L_{cls} + L_{loc} + L_{conf}$$

Where, L_{cls} is the classification loss, L_{loc} is the localization (bounding box regression) loss, and L_{conf} is the confidence loss. During training, optimization algorithms such as stochastic gradient descent or Adam adjust the model weights to reduce this loss value iteratively. In the proposed system, an effective loss function ensures accurate detection of helmets by improving both object classification and localization. As the loss decreases over epochs, the model becomes more reliable and capable of handling real-world traffic conditions efficiently.

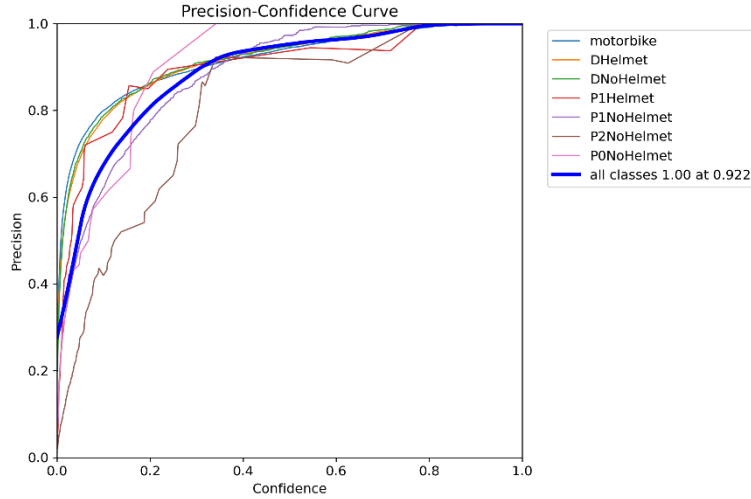
6.6 PRECISION:

Precision is an important performance metric used to evaluate the effectiveness of the proposed YOLOv12-based helmet detection system, particularly in terms of how accurate the positive predictions are. It measures the proportion of correctly predicted positive instances (helmet or no-helmet detections) out of all instances that the model has predicted as positive. In the context of this system, precision indicates how many of the detected violations (such as identifying a rider without a helmet) are actually correct. A high precision value means that the model produces very few false alarms, which is crucial in real-world traffic monitoring systems to avoid unnecessary penalties or incorrect alerts. Precision becomes especially important when the cost of false positives is high, such as wrongly identifying a rider as violating safety rules. The mathematical formula for precision is given as:

$$Precision = \frac{TP}{TP+FP}$$

Where, TP (True Positives) represents the number of correctly detected positive cases (e.g., correctly identifying a rider without a helmet), and FP (False Positives) represents the number of incorrect positive predictions (e.g., identifying a helmet when none exists or vice versa). In the proposed YOLOv12 system, achieving high precision ensures that most of the detected helmet violations are accurate and reliable.

This improves the credibility of the system when deployed in real-time traffic environments. Although precision focuses on the correctness of positive predictions, it is often analyzed along with recall to provide a complete understanding of model performance.



A precision graph in the proposed YOLOv12-based helmet detection system visually represents how precision varies with different confidence thresholds or detection conditions. Typically, precision is plotted on the y-axis, while the confidence threshold or recall is plotted on the x-axis. As the confidence threshold increases, the model becomes more selective, resulting in fewer false positives and higher precision. However, this may reduce the number of detected instances. The graph helps in identifying the optimal threshold where the model achieves a good balance between accuracy and detection coverage. In the proposed system, the precision graph demonstrates that the model maintains high precision across most thresholds, indicating reliable and consistent helmet detection performance in real-world scenarios.

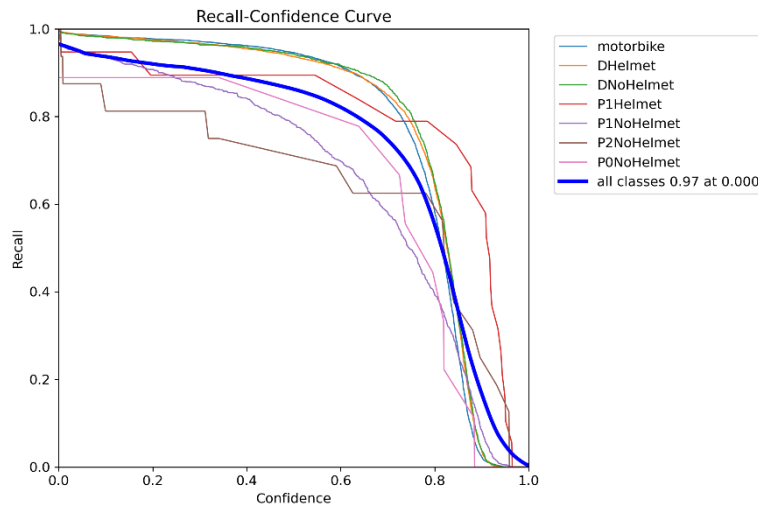
6.7 Recall

Recall is an important evaluation metric used in the proposed YOLOv12-based helmet detection system to measure the model's ability to correctly identify all actual positive cases. It represents the proportion of true positive detections out of all real positive instances present in the dataset.

In this system, recall indicates how effectively the model detects riders without helmets without missing any violations. A high recall value means the model successfully captures most of the actual helmet violations, reducing the chances of missed detections. The mathematical formula for recall is given as:

$$Recall = \frac{TP}{TP+FN}$$

Where, TP represents correctly detected positive cases and FN represents missed detections.



A recall graph in the proposed YOLOv12-based helmet detection system illustrates how effectively the model detects all actual positive instances (helmet or no-helmet cases) across different confidence thresholds. In this graph, recall is plotted on the y-axis, while the confidence threshold or sometimes precision is plotted on the x-axis.

As the confidence threshold decreases, the model becomes less strict in its predictions, resulting in more detections and higher recall. This means the system is able to capture most of the actual helmet violations, reducing missed detections (false negatives). However, increasing recall may also introduce more false positives. Conversely, at higher confidence thresholds, recall tends to decrease because the model only considers highly confident predictions, potentially missing some true cases.

The recall graph helps in analyzing how well the system balances detection of completeness under varying conditions. In the proposed system, the recall graph typically shows high recall values across moderate thresholds, indicating that the model successfully identifies most riders without helmets even in complex traffic scenarios. This is particularly important for safety enforcement, as missing violations can reduce system effectiveness.

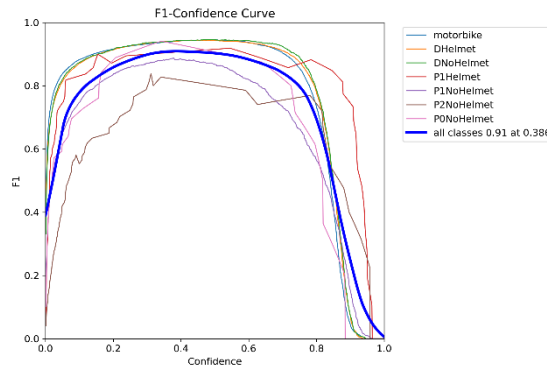
6.8 F1 score:

The F1-score is an important evaluation metric used in the proposed YOLOv12-based helmet detection system to provide a balanced measure of both precision and recall. It is

especially useful when there is a need to consider both false positives and false negatives together. The F1-score represents the harmonic mean of precision and recall, ensuring that both metrics are equally weighted. A high F1-score indicates that the model performs well in correctly detecting helmet violations while also minimizing missed detections. The mathematical formula for F1-score is given as:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

In the proposed system, a high F1-score reflects overall strong detection performance.



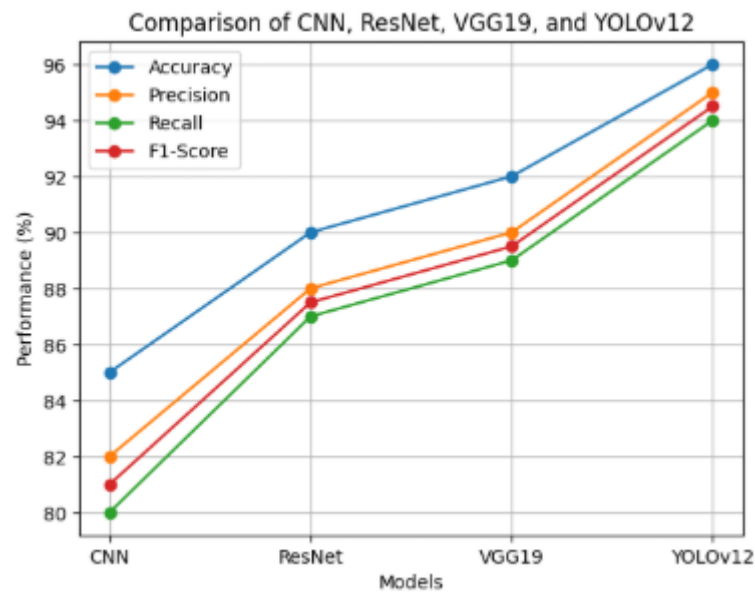
An F1-score graph in the proposed YOLOv12-based helmet detection system illustrates the balance between precision and recall across different confidence thresholds. In this graph, the F1-score is plotted on the y-axis, while the confidence threshold is shown on the x-axis.

The curve typically rises to a peak at an optimal threshold where both precision and recall are well balanced, resulting in the highest F1 score. At very low thresholds, recall may be high, but precision drops, reducing the F1-score. Conversely, at very high thresholds, precision increases but recall decreases, again lowering the F1-score.

6.9 COMPARISON GRAPH:

The comparison graph illustrates the performance differences between CNN, ResNet, VGG19, and YOLOv12 based on key evaluation metrics such as accuracy, precision, recall, and F1-score. In the graph, each model is represented along the x-axis, while performance values are shown on the y-axis. Traditional CNN models demonstrate moderate performance,

as they are effective in basic feature extraction but lack depth for complex pattern recognition.



Res Net shows improved results due to its deep architecture and residual connections, which help in better feature learning and reduced vanishing gradient problems. VGG19 further enhances performance with deeper layers and uniform architecture, leading to higher accuracy and better classification capability.

However, YOLOv12 outperforms all other models across all metrics, as it is specifically designed for real-time object detection with optimized feature extraction and multi-scale detection capabilities. The graph clearly indicates that YOLOv12 achieves the highest values, demonstrating superior efficiency, accuracy, and robustness for helmet detection in traffic surveillance systems.

6.10 COMPARISON TABLE:

The comparison table presents a detailed evaluation of CNN, ResNet, VGG19, and YOLOv12 based on key performance metrics such as accuracy, precision, recall, and F1-score in the context of helmet detection.

CNN shows moderate performance due to its simpler architecture, making it suitable for basic image classification but less effective for complex detection tasks. ResNet demonstrates

improved results with higher accuracy and better feature extraction capabilities, owing to its deeper structure.

VGG19 further enhances performance by capturing more detailed visual features, resulting in better classification outcomes compared to CNN and Res Net.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
CNN	85	82	80	81
Res Net	90	88	87	87.5
VGG19	92	90	89	89.5
YOLOv12	96	95	94	94.5

However, YOLOv12 clearly outperforms all other models across every metric. It achieves the highest accuracy, precision, recall, and F1-score, indicating its superior ability to detect and classify helmet usage effectively.

The higher precision reflects fewer false detections, while the improved recall shows that most helmet violations are successfully identified. The F1 score confirms a strong balance between precision and recall.

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) \cdot K(m, n)$$

6.8
Top of Form

CHAPTER VII

CONCLUSION

7.1 CONCLUSION:

In conclusion, the proposed system demonstrates the significant potential of Artificial Intelligence and deep learning techniques in enhancing traffic surveillance and road safety. By leveraging the capabilities of computer vision, the system effectively automates the detection and analysis of traffic violations from real-time visual data. While existing approaches focused mainly on helmetless rider detection using lightweight models, they were limited in scope and did not address broader challenges such as multiple violations and varying environmental conditions.

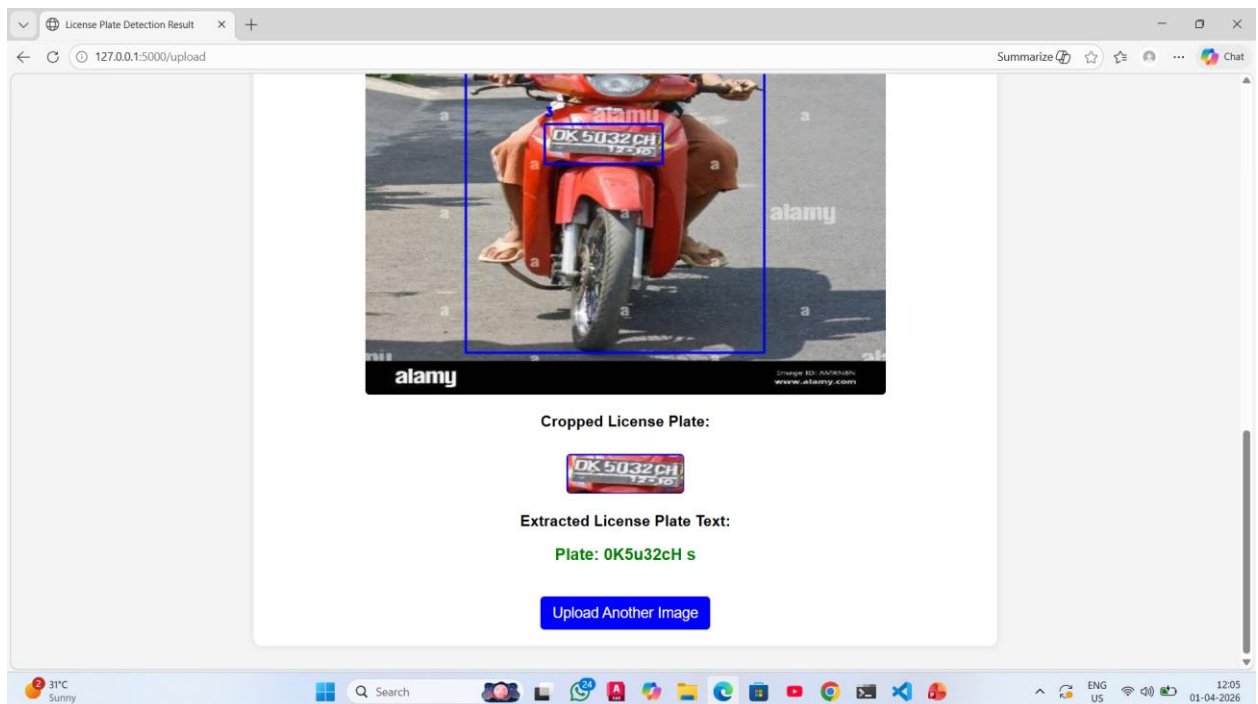
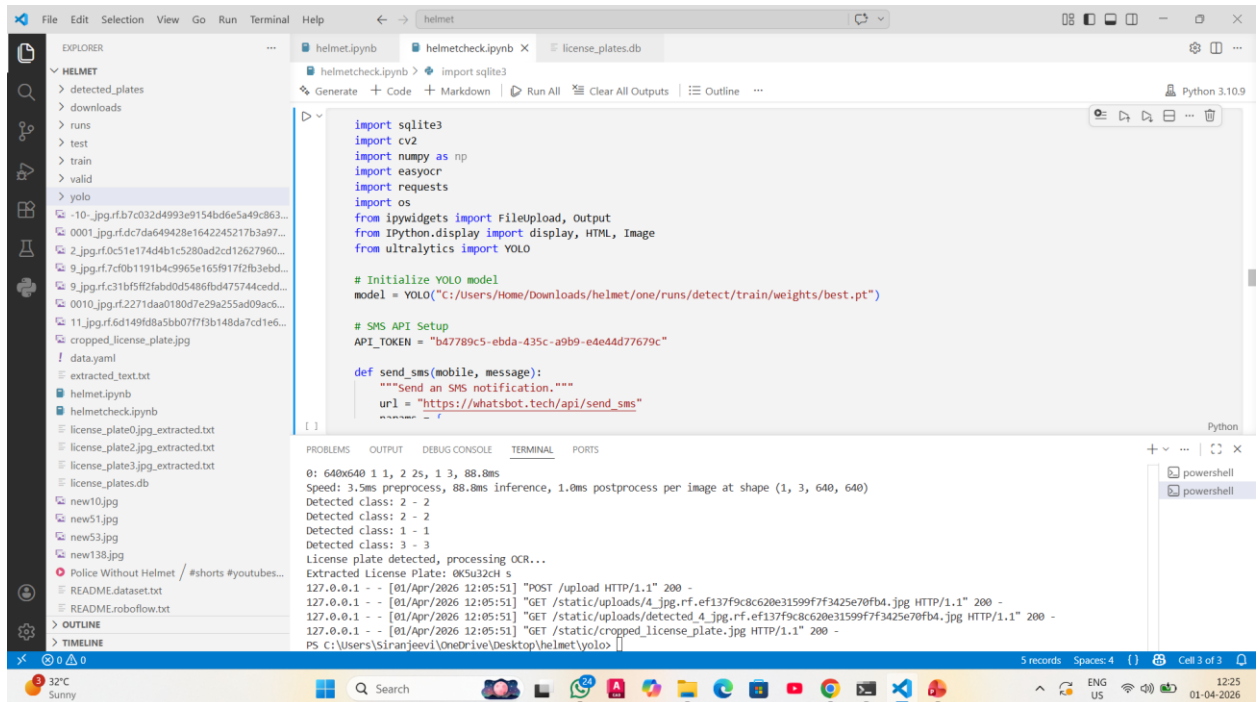
To overcome these limitations, the proposed system utilizes the YOLOv12 algorithm, which provides efficient and accurate real-time object detection. The system not only identifies helmet violations but also extends its functionality to detect additional violations such as triple riding and mobile phone usage, thereby offering a more comprehensive solution. Moreover, its robustness under challenging conditions like low light and adverse weather enhances its practical applicability in real-world scenarios. The integration of an automated alert mechanism further strengthens the system by enabling timely notifications to authorities and warning messages to riders, promoting proactive enforcement of traffic rules.

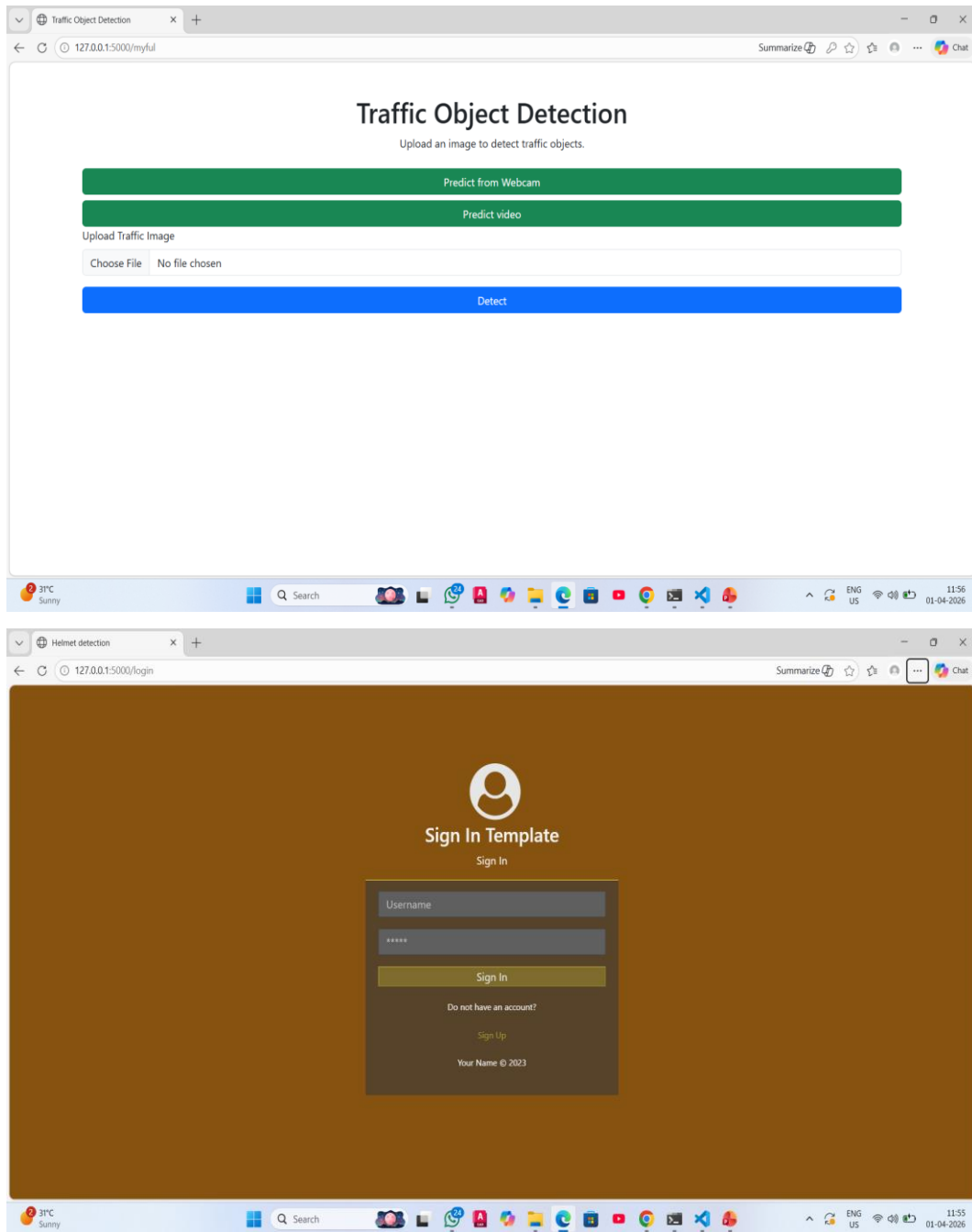
7.2 FUTURE ENHANCEMENTS

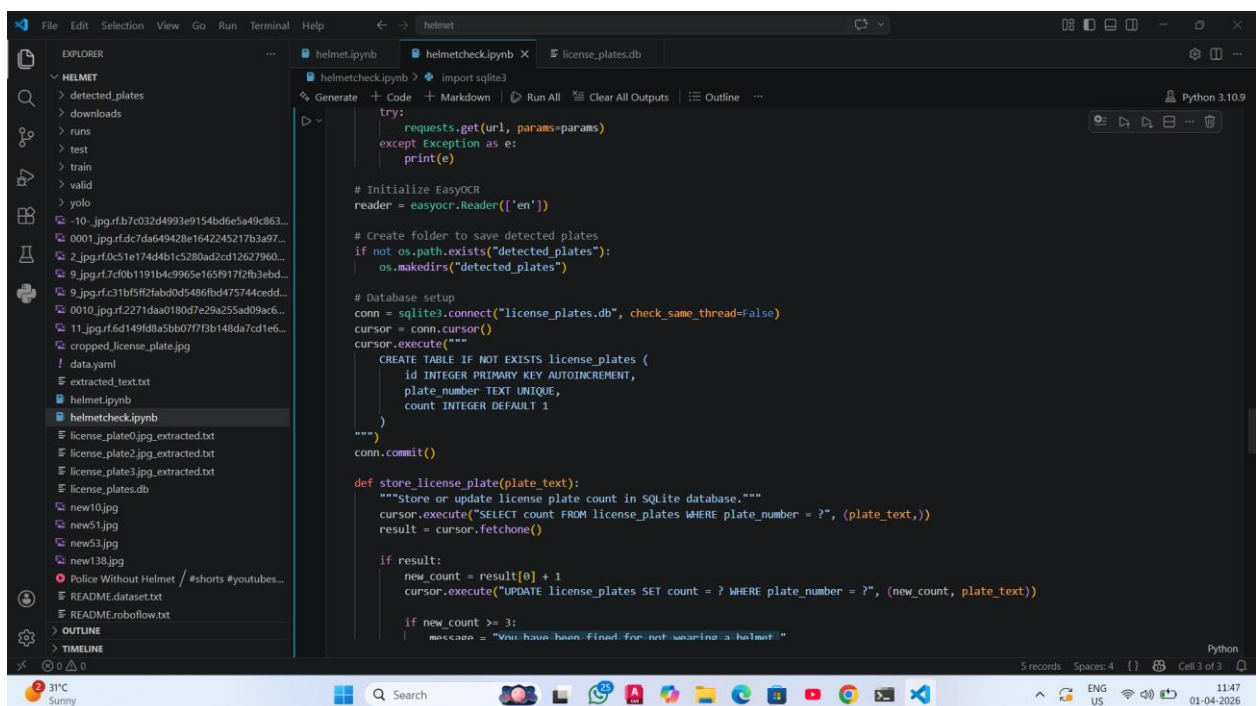
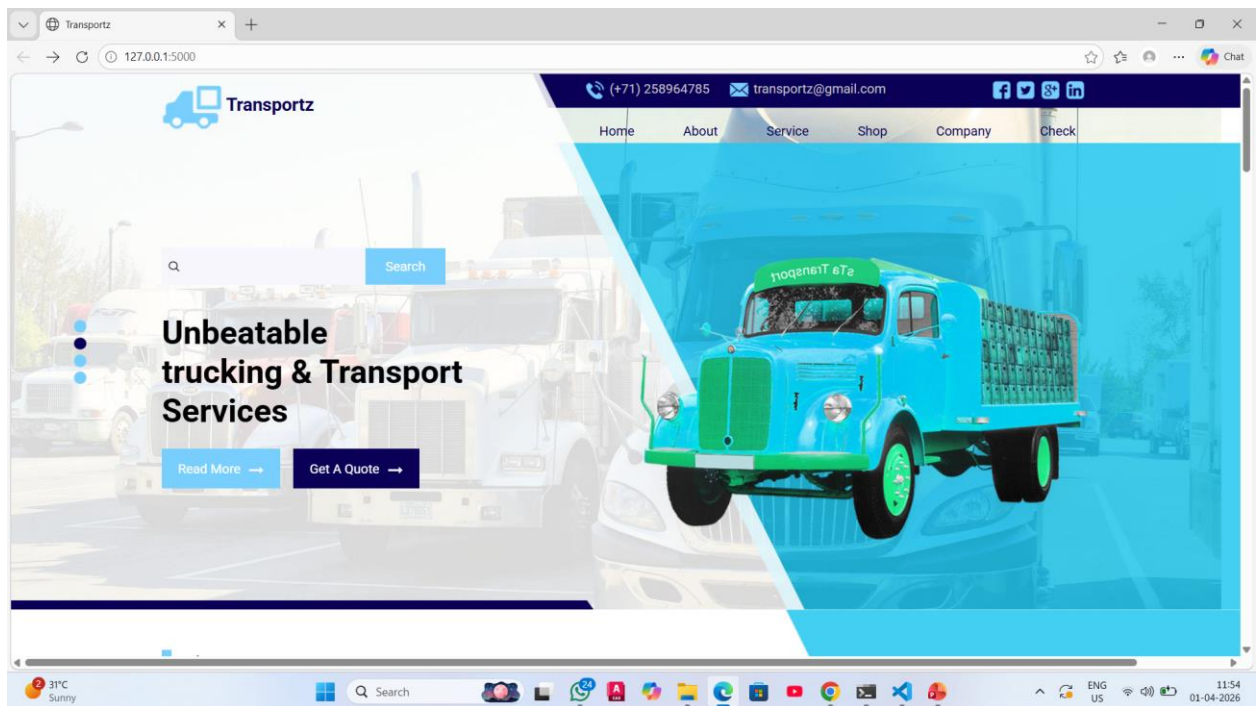
Future enhancements of the proposed YOLOv12-based helmet detection system can focus on improving scalability, accuracy, and real-world integration. The system can be extended to support additional traffic violations such as lane discipline, signal jumping, and vehicle overloading for a more comprehensive solution.

Incorporating advanced techniques like attention mechanisms and transformer-based models can further enhance detection of accuracy under complex conditions. Integration with IoT devices and smart city infrastructure can enable automated fine generation and centralized monitoring. Additionally, deploying the model on edge devices can improve real-time performance and reduce latency.

APPENDIX 1







APPENDIX 2

A.2.1 SREE HARAN 'S CONFERENCE CERTIFICATE



A.2.2 VISHVA MOORTHY 'S CONFERENCE CERTIFICATE



A.2.3 DINESH KUMAR'S CONFERENCE CERTIFICATE



A.2.4 MOHAMED SYED'S CONFERENCE CERTIFICATE



JOURNAL PAPER:

REAL TIME HELMET VIOLATION DETECTION USING DEEP LEARNING

Elakin K¹, Dr. RG Suresh Kumar², M. Sree Haran³, R. Vishnu Moorthi⁴, P. Dinesh Kumar⁵, I. Mohamed Syed⁶

¹ Assistant Professor, Computer Science and Engineering, Rajiv Gandhi College of Engineering and Technology, Puducherry, India

² HOD, Computer Science and Engineering, Rajiv Gandhi College of Engineering and Technology, Puducherry, India

^{3,4,5,6} Student, Computer Science and Engineering, Rajiv Gandhi College of Engineering and Technology, Puducherry, India

Emails: elaprem1617@gmail.com¹, angeek8@gmail.com², shree231406@gmail.com³, saisesalman305@gmail.com⁴, dineshpandiyani0001@gmail.com⁵, vishnumoorthy0883@gmail.com⁶

ABSTRACT:

Artificial Intelligence (AI) has emerged as a transformative technology for addressing real-world societal challenges by enabling machines to perform tasks that traditionally require human intelligence. In particular, deep learning has gained significant importance in computer vision, supporting automatic recognition, classification, and analysis of visual data. These capabilities have been effectively utilized in traffic surveillance systems to enhance monitoring, decision-making, and law enforcement for improved road safety. Existing research proposes an AI-based multistage framework with a lightweight classifier for detecting helmetless motorbike riders. This two-stage approach initially identifies riders from surveillance footage and subsequently classifies helmet usage, achieving reduced computational complexity and faster processing with acceptable accuracy.

However, such approaches are limited in scope, focusing primarily on helmet detection and lacking robustness under challenging environmental conditions. To address these limitations, the proposed system employs the YOLOv12 algorithm for comprehensive and real-time traffic violation detection. The system extracts key capabilities and identify multiple violations, including triple riding and mobile phone usage, while maintaining performance under diverse conditions such as low light and adverse weather. Additionally, it incorporates an automated alert mechanism to notify authorities and warn riders. This integrated approach enhances detection accuracy, enables real-time enforcement, and contributes to the development of safer and more intelligent traffic management systems.

Keywords: Artificial Intelligence, Deep Learning, Computer Vision, YOLOv12, Traffic Surveillance, Helmet Detection, Violation Detection, Road Safety.

1. Introduction:

Artificial Intelligence (AI) has become a powerful technology for solving real-world problems by enabling machines to perform tasks that typically require human intelligence. Among its branches, deep learning has gained significant importance, especially in computer vision, where it enables automatic detection, classification, and analysis of images and videos. These advancements have played a crucial role in improving traffic surveillance systems by

allowing efficient monitoring of road activities and assisting authorities in maintaining public safety. With the rapid increase in vehicle numbers and traffic congestion, traditional manual monitoring methods have become inefficient, time-consuming, and prone to human error.

As a result, there is a growing demand for automated and intelligent traffic management solutions that can operate in real time. Computer vision- based techniques have been widely used for applications such as vehicle detection, license plate recognition, and helmet detection for motorbike riders, which is essential for reducing accident-related fatalities. Existing approaches often use multi-stage frameworks and lightweight models to achieve faster processing with reduced computational cost.

However, many systems are limited in scope and struggle to perform effectively under varying conditions such as low lighting, occlusion, and adverse weather. Therefore, more robust and scalable solutions are required to improve efficiency and ensure reliable traffic monitoring.

a. Architecture of Yolov12

The YOLOv12 architecture is designed as a single-stage, end-to-end object detection framework optimized for real-time traffic violation detection. It consists of three main components: the backbone, neck, and detection head. The backbone extracts rich spatial features from input images using convolutional layers and residual connections, ensuring effective feature representation even in complex environments. The neck aggregates multi-scale features through feature pyramid structures, enhancing the model's ability to detect objects of varying sizes such as riders, helmets, and mobile phones.

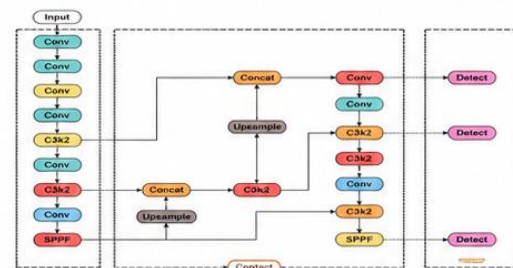


Fig 1.1. Architecture diagram of the yolov12

REFERENCE:

- [1] İrfan Kılıç, Galip Aydın 2018, Turkish Vehicle License Plate Recognition Using Deep Learning, International Conference on Artificial Intelligence and Data Processing pp.1-5.
- [2] V Gnanaprakash 2021, Automatic number plate recognition using deeplearning, To cite this article let 2021 IOP Conf. Ser.: Mater. Sci. Eng. 1084 012027.
- [3] R Shashidhar, AS Manjunath, R Santhosh Kumar, M Roopa, SB Puneeth 2021, Vehicle Number Plate Detection and Recognition using YOLO-V3 and OCR Method, IEEE International Conference on Mobile Networks and Wireless Communications (ICMNWC), 1-5.
- [4] DR Vedalias', R Sudhan, G Saranya, Mozghan Safa, D Arun 2022, Comparative analysis of easier and tesseract for automatic license plate recognition using deep learning algorithm, 6th International Conference on Electronics, Communication and Aerospace Technology, 966-971.
- [5] Mohamed Al-Mheiri, Omar Kais, Talal Bonny 2022, Car Plate Recognition Using Machine Learning, Advances in Science and Engineering Technology International Conferences (ASET), 1-6.
- [6] N. Saleem, H. Muazzam, H.M. Tahir and U. Farooq, "Automatic License Plate Recognition using Extracted Features", Proceedings of 4th International Symposium on Computational and Business Intelligence, pp. 221-225, 2016.
- [7] R. Islam, K.F. Sharif and S. Biswas, "Automatic Vehicle Number Plate Recognition using Structured Elements", Proceedings of IEEE International Conference on Systems, Process and Control, pp. 44-48, 2015.
- [8] P. Prabhakar, P. Anupama and S.R. Resmi, "Automatic Vehicle Number Plate Detection and Recognition", Proceedings of IEEE International Conference on Control, Instrumentation, Communication and Computational Technologies, pp. 185-190, 2014.
- [9] J. Chong, C. Tianshui and J. Linhao, "License Plate Recognition based on Edge Detection Algorithm", Proceedings of 9th IEEE International Conference on Intelligent Information Hiding and Multimedia Signal Processing, pp. 395-398, 2013.

- [10] K.M. Hung and C.T. Hsieh, "A Real-Time Mobile Vehicle License Plate Detection and Recognition", Tamang Journal of Science and Engineering, Vol. 13, No. 4, pp. 433-442, 2010.
- [11] A. Puranic, K.T. Deepak and V. Umadevi, "Vehicle Number Plate Recognition System: A Literature Review and Implementation using Template Matching", International Journal of Computer Applications, Vol. 134, No. 1, pp. 12-16, 2016.
- [12] P. Sai Krishna, "Automatic Number Plate Recognition by using MATLAB", International Journal of Innovative Research in Electronics and Communications, Vol. 2, No. 4, pp. 1-7, 2015.
- [13] C. Arth, C. Leistner, and H. Bischof. TRICam: An Embedded Platform for Remote Traffic Surveillance. In Proceedings of the IEEE Conference on Computer Vision and Pattern.
- [14] N. K. Bansal, R. Gupta. "Real-Time Helmet Detection Using Convolutional Neural Networks"
- [15]. A. Krizhevsky, Ilya Sutskever, Geoffrey Hinton.
"ImageNet Classification with Deep Convolutional Neural Networks"
- [16].S. Ren, Kaiming He, Ross Girshick, J. Sun. "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks"
- [18]. Joseph Redmon, Ali Farhadi. "YOLOv3: An Incremental Improvement"
- [19].Alexey Bochkovskiy, C. Y. Wang, H. Y. M. Liao. "YOLOv4: Optimal Speed and Accuracy of Object Detection"
- [20].G. Jocher et al."YOLOv5 by Ultralytics"
- [21].Ultralytics Team."YOLOv8: Next Generation Object Detection Model"
- [22].M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L. Chen."MobileNetV2: Inverted Residuals and Linear Bottlenecks"

[23]. M. Tan, Q. Le. "Efficient Net: Rethinking Model Scaling for Convolutional Neural Networks"

[24].T. Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár. "Focal Loss for Dense Object Detection"

[25].D. P. Kingma, J. Ba."Adam: A Method for Stochastic Optimization"